

Familiarity with gcc

Gnu C compiler (gcc) provides a large number of options (or switches) to the user for controlling the compilation process. This document is a brief introduction to some of the options, particularly to those related to vectorization and parallelization capabilities of gcc.

Outline of Compilation Options in gcc

S N	Option	Effect
1	-c	Compile or assemble the source files, but do not link. The linking stage is not performed. The ultimate output is in the form of an object file (.o) for each source file.
2	-S	Stop after the stage of compilation and generate assembly code but do not assemble it to produce machine code. The output is in the form of an assembler code file for each non- assembler input file specified. By default, the assembled file name for a source file is made by replacing the suffix .c with .s
3	-E	Stop after the preprocessing stage; do not run the compiler proper. The o/p is in the form of preprocessed source code, which is sent to the stdout. Useful if we wish to the effect of the c-preprocessor on our c-source.
4	-o filename	This option results in the output being produced in "filename" and applies all output produced by gcc, be an executable file, an object file, an assembler file or preprocessed C code. If -o is not specified, the default is to put an executable file in "a.out", the object file for source.suffix in source.o, its assembler file in source.s, a precompiled header file in source.suffix.gch, and all preprocessed C source on standard output.
5	-v	Verbose option : Print (on standard error output) the commands executed to run the stages of compilation. Also print the version number of the compiler driver program and of the preprocessor and the compiler proper.
6	@file	Read command-line options from file. The options read are inserted in place of the original @file option. If file does not exist, or cannot be read, then the option will be treated literally, and not removed. Options in file are separated by whitespace. A whitespace character may be included in an option by surrounding the entire option in either single or double quotes. Any character (including a backslash) may be included by prefixing the character to be included with a backslash. The file may itself contain additional @file options; any such options will be processed recursively.
7	-O_c	Enables optimization to be performed by gcc. The character 'c' may be one of {null, 0, 1, 2, 3, s}. The option -O0, where c is null is the default and informs gcc not to enable optimizations. The option -O1 or -O are identical and few optimizations are enabled with this switch. The option -O2 leads to more vigorous optimizations to be performed. The option -O3 is the highest level of optimization permissible. The option -Os enables optimizations for space. Use man gcc or gcc -v option to get details of optimizations that are enabled with each switch.
8	-fdump-tree-all	Shows the dump of compilation passes. The number of dumps vary depending on the other options such as optimization levels. Without optimization, this option displays the dumps of a few passes. Among the ones listed, the following 5 are often useful, i) source.c.004t.gimple which gives the equivalent gimple code after the 4 th pass, gimple is the Intermediate Representation (IR) used by gcc, ii) source.c.014t.cfg which is the control flow graph constructed from gimple code in the 14 th pass, iii) source.c.018t.ssa which gives ssa code after the 18 th pass, iv) source.c.149t.optimized which is the optimized code produced after the 149 th pass, and v) source.c.232t.cfg which gives statistics about the optimizations performed by gcc. With -O2 the compiler produces a large number of dumps as more passes are enabled in this option.

S. N.	Useful Combination of gcc options	Output(s) produced by gcc
1.	-O2 -fverbose-asm -S	Assembled code in verbose mode with optimizaion level O2 enabled. The executable is not created.
2.	-O2 -fverbose-asm -S -fdump-tree-all	Dumps of various passes alongwith the dumps given out by option 1 above.
3.	-O2 -ftree-vectorize -fdump-tree-vect-all -msse4	To generate executable vector code with minimal dumps.
4.	-O2 -ftree-parallelize-loops=4 -fdump-tree-parloops -ftree-loop-distribution -fdump-tree-ldist	To produce executable parallel OMP threads with minimal dumps.

I